

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Food Store

Sistema de Gestión de Pedidos

Materia: Base de Datos I

Integrante:

- Alfredo Castillo – Comisión 3
- Cesia Castilla – Comisión 5
- Rocio Agüero – Comisión 6

Universidad Tecnológica Nacional.

Carrera: (TUPaD)

Fecha de entrega: Junio 2026

Indice

Resumen ejecutivo	3
Reglas de negocios.....	3
Diagrama entidad - relacion	4
Modelo relacional.....	4
Evidencias	5
Etapa 1	5
Etapa 2	7
Etapa 3	9
Etapa 4	13
Etapa 5	15
Evidencia de uso de IA como tutora	19
Anexo IA –Etapa 1	19
Anexo IA –Etapa 2	21
Anexo IA –Etapa 3.....	22
Anexo IA –Etapa 4	23
Anexo IA –Etapa 5	24
Bibliografía.....	27
Link de video	28

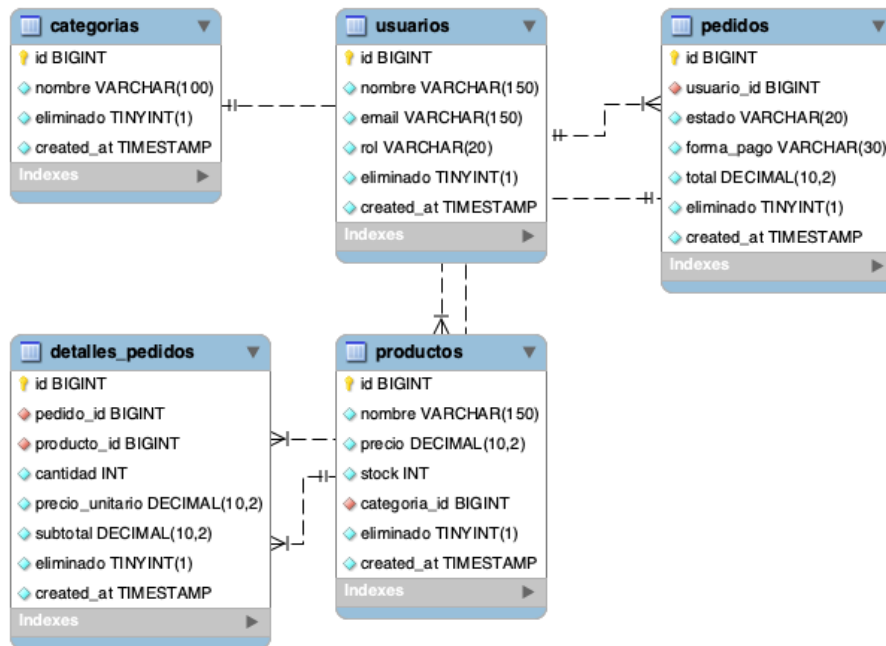
Resumen Ejecutivo

En este trabajo se presenta el ciclo de desarrollo de la base de datos `food_store`, iniciando con un modelo Entidad-Relación que asegura la integridad de los datos desde el esquema. Mediante técnicas avanzadas de SQL, se realizó una simulación de carga masiva para evaluar el rendimiento ante consultas complejas, uniones de tablas y la creación de vistas. El diseño final destaca por la inclusión de medidas de seguridad para restringir accesos no autorizados y por un análisis crítico de la concurrencia, donde se validó el comportamiento transaccional y los niveles de aislamiento óptimos frente a escenarios de alta demanda simultánea.

Reglas de negocio.

- CATEGORIA: Identidad Única (No pueden existir dos categorías del mismo nombre)
- USUARIO: Identidad Única (No puede haber dos usuarios con el mismo correo electrónico)
- PRODUCTOS: Todo producto debe permanecer a una categoría existente, el precio del producto no puede ser negativo y el stock disponible no puede ser menor a cero.
- PEDIDOS: Un pedido debe estar asociado a un usuario registrado, el estado del pedido está restringido a cuatro tipos, solo acepta tres formas de pago y el valor total no puede ser negativo.
- DETALLE PEDIDO: Si el pedido principal es eliminado, todos sus detalles se borran automáticamente.
- BORRADO LOGICO: El sistema no elimina físicamente los registros, se utiliza una bandera para marcar la baja.

Diagrama Entidad-Relación (DER).



Modelo Relacional (MR).

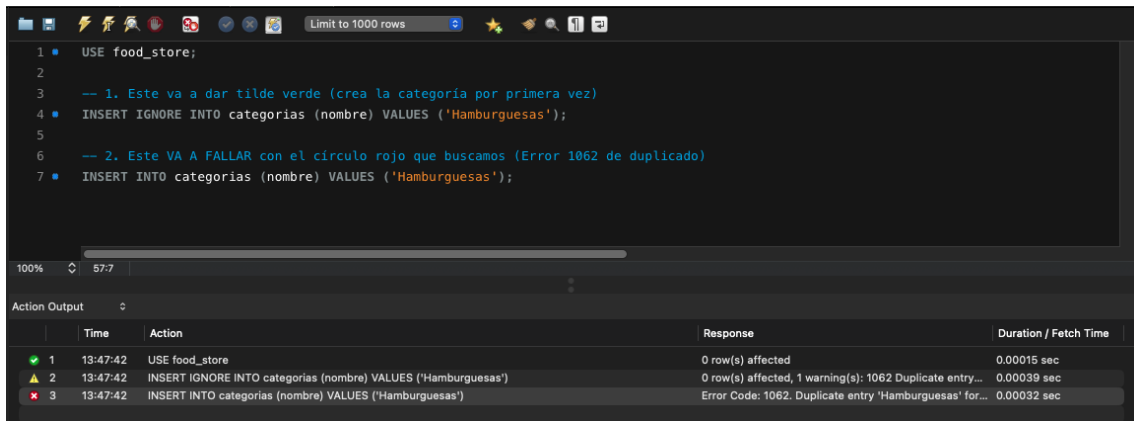
- Categorías(id, nombre, eliminado, created_at)
- Usuarios(id, nombre, email, rol, eliminado, created_at)
- Productos(id, nombre, precio, stock, categoría_id, eliminado, created_at)
- Pedidos(id, usuario_id, estado, forma_pago, total, eliminado, created_at)
- Detalles_pedidos(id, pedido_id, producto_id, cantidad, precio_unitario, subtotal, eliminado, created_at)

Evidencia de Etapa 1 – Modelado y constrains.

Caso 1: Violación de Restricción UNIQUE (Nombre Duplicado)

Referencia: (ver 01 esquema.sql y 02_catalogos.sql)

Al no tener descripción, solo le pasamos el nombre:

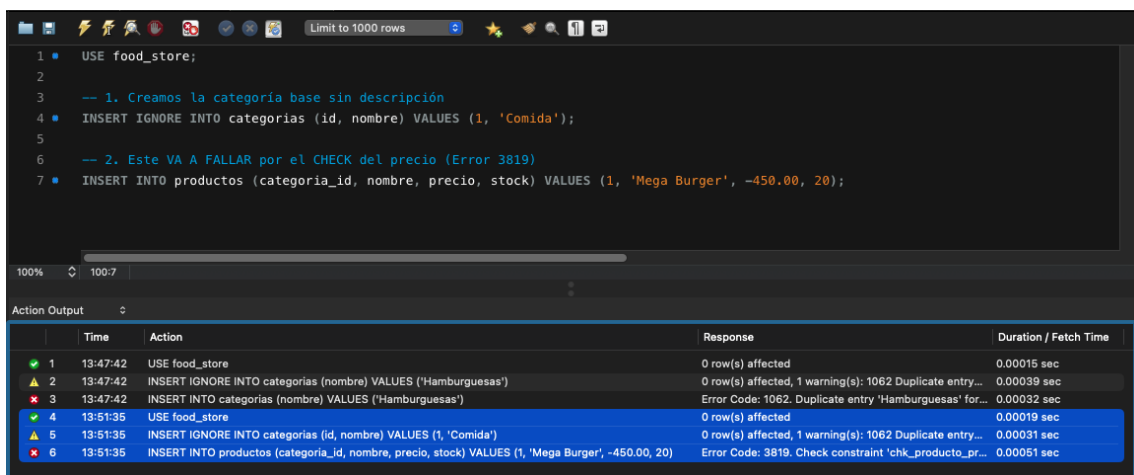


```
1 • USE food_store;
2
3 -- 1. Este va a dar tilde verde (crea la categoría por primera vez)
4 • INSERT IGNORE INTO categorias (nombre) VALUES ('Hamburguesas');
5
6 -- 2. Este VA A FALLAR con el círculo rojo que buscamos (Error 1062 de duplicado)
7 • INSERT INTO categorias (nombre) VALUES ('Hamburguesas');
```

	Time	Action	Response	Duration / Fetch Time
✓ 1	13:47:42	USE food_store	0 row(s) affected	0.00015 sec
⚠ 2	13:47:42	INSERT IGNORE INTO categorias (nombre) VALUES ('Hamburguesas')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00039 sec
✗ 3	13:47:42	INSERT INTO categorias (nombre) VALUES ('Hamburguesas')	Error Code: 1062. Duplicate entry 'Hamburguesas' for...	0.00032 sec

Caso 2: Violación de Restricción CHECK (Precio Negativo)

Primero nos aseguramos de que exista la categoría con ID 1 usando tu estructura limpia, y luego tiramos el precio negativo:



```
1 • USE food_store;
2
3 -- 1. Creamos la categoría base sin descripción
4 • INSERT IGNORE INTO categorias (id, nombre) VALUES (1, 'Comida');
5
6 -- 2. Este VA A FALLAR por el CHECK del precio (Error 3819)
7 • INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (1, 'Mega Burger', -450.00, 20);
```

	Time	Action	Response	Duration / Fetch Time
✓ 1	13:47:42	USE food_store	0 row(s) affected	0.00015 sec
⚠ 2	13:47:42	INSERT IGNORE INTO categorias (nombre) VALUES ('Hamburguesas')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00039 sec
✗ 3	13:47:42	INSERT INTO categorias (nombre) VALUES ('Hamburguesas')	Error Code: 1062. Duplicate entry 'Hamburguesas' for...	0.00032 sec
✓ 4	13:51:35	USE food_store	0 row(s) affected	0.00019 sec
⚠ 5	13:51:35	INSERT IGNORE INTO categorias (id, nombre) VALUES (1, 'Comida')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00031 sec
✗ 6	13:51:35	INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (1, 'Mega Burger', -450.00, 20)	Error Code: 3819. Check constraint 'chk_producto_pr...	0.00051 sec

Caso 3: Violación de Restricción CHECK (Stock Negativo)

Este va a buscar la categoría 1 que creamos recién y va a saltar por el stock menor a cero:

```
1 USE food_store;
2
3 -- 1. Creamos la categoría base sin descripción
4 INSERT IGNORE INTO categorias (id, nombre) VALUES (1, 'Comida');
5
6 -- 2. Este va a FALLAR por el CHECK del precio (Error 3819)
7 INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (1, 'Mega Burger', -450.00, 20);
```

	Time	Action	Response	Duration / Fetch Time
1	13:47:42	USE food_store	0 row(s) affected	0.00015 sec
2	13:47:42	INSERT IGNORE INTO categorias (nombre) VALUES ('Hamburguesas')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00039 sec
3	13:47:42	INSERT INTO categorias (nombre) VALUES ('Hamburguesas')	Error Code: 1062. Duplicate entry 'Hamburguesas' for...	0.00032 sec
4	13:51:35	USE food_store	0 row(s) affected	0.00019 sec
5	13:51:35	INSERT IGNORE INTO categorias (id, nombre) VALUES (1, 'Comida')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00031 sec
6	13:51:35	INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (1, 'Mega Burger', -450.00, 20)	Error Code: 3819. Check constraint 'chk_producto_pr...	0.00051 sec
7	13:52:53	USE food_store	0 row(s) affected	0.00015 sec
8	13:52:53	INSERT IGNORE INTO categorias (id, nombre) VALUES (1, 'Comida')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00017 sec
9	13:52:53	INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (1, 'Mega Burger', -450.00, 20)	Error Code: 3819. Check constraint 'chk_producto_pr...	0.00020 sec

Caso 4: Violación de Clave Foránea FOREIGN KEY (ID Inexistente)

Este apunta a la categoría 999 que no existe en tu sistema:

```
1 USE food_store;
2
3 INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (999, 'Producto Huérfano', 1500.00, 10);
```

	Time	Action	Response	Duration / Fetch Time
1	13:47:42	USE food_store	0 row(s) affected	0.00015 sec
2	13:47:42	INSERT IGNORE INTO categorias (nombre) VALUES ('Hamburguesas')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00039 sec
3	13:47:42	INSERT INTO categorias (nombre) VALUES ('Hamburguesas')	Error Code: 1062. Duplicate entry 'Hamburguesas' for...	0.00032 sec
4	13:51:35	USE food_store	0 row(s) affected	0.00019 sec
5	13:51:35	INSERT IGNORE INTO categorias (id, nombre) VALUES (1, 'Comida')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00031 sec
6	13:51:35	INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (1, 'Mega Burger', -450.00, 20)	Error Code: 3819. Check constraint 'chk_producto_pr...	0.00051 sec
7	13:52:53	USE food_store	0 row(s) affected	0.00015 sec
8	13:52:53	INSERT IGNORE INTO categorias (id, nombre) VALUES (1, 'Comida')	0 row(s) affected, 1 warning(s): 1062 Duplicate entry...	0.00017 sec
9	13:52:53	INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (1, 'Mega Burger', -450.00, 20)	Error Code: 3819. Check constraint 'chk_producto_pr...	0.00020 sec
10	13:54:54	USE food_store	0 row(s) affected	0.00014 sec
11	13:54:54	INSERT INTO productos (categoria_id, nombre, precio, stock) VALUES (999, 'Producto Huérfano', 1500...	Error Code: 1452. Cannot add or update a child row: a...	0.00063 sec

Evidencia de Etapa 2 – Implementacion y carga

Referencia: (ver 03_carga_masiva.sql)

Consulta que muestra el volumen total de registros insertados.

```
110 • SELECT 'usuarios' AS tabla, COUNT(*) AS total_registros FROM usuarios
111 UNION ALL
112 SELECT 'productos', COUNT(*) FROM productos
113 UNION ALL
114 SELECT 'pedidos', COUNT(*) FROM pedidos
115 UNION ALL
116 SELECT 'detalles_pedidos', COUNT(*) FROM detalles_pedidos;
```

tabla	total_registros
usuarios	1003
productos	1000
pedidos	4000
detalles_pedidos	10000

Muestra de productos con categoría invalida.

```
119 • SELECT COUNT(*) AS productos_con_categoria_invalida
120 FROM productos p LEFT JOIN categorias c ON c.id = p.categoria_id
121 WHERE c.id IS NULL;
```

productos_con_categoria_invalida
0

Muestra de pedidos con usuarios inválidos.

```
123 • SELECT COUNT(*) AS pedidos_con_usuario_invalido
124 FROM pedidos p LEFT JOIN usuarios u ON u.id = p.usuario_id
125 WHERE u.id IS NULL;
```

pedidos_con_usuario_invalido
0

Muestra de detalles con pedidos inválidos.

```
127 • SELECT COUNT(*) AS detalles_con_pedido_invalido
128 FROM detalles_pedidos d LEFT JOIN pedidos p ON p.id = d.pedido_id
129 WHERE p.id IS NULL;
```

detalles_con_pedido_invalido
0

Muestra detalles con productos inválidos.

```
131 • SELECT COUNT(*) AS detalles_con_producto_invalido
132 FROM detalles_pedidos d LEFT JOIN productos pr ON pr.id = d.producto_id
133 WHERE pr.id IS NULL;
134
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
detalles_con_producto_invalido				
	0			

Cardinalidad minima: pedidos sin ningún detalle

```
136 • SELECT COUNT(*) AS pedidos_sin_detalle
137 FROM pedidos p
138 LEFT JOIN detalles_pedidos d ON d.pedido_id = p.id
139 WHERE d.id IS NULL;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
pedidos_sin_detalle				
	0			

Rango de dominio fuera de lo esperado.

```
142 • SELECT COUNT(*) AS productos_fuera_de_rango FROM productos WHERE precio < 0 OR stock < 0;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
productos_fuera_de_rango				
	0			

```
143 • SELECT COUNT(*) AS detalles_fuera_de_rango FROM detalles_pedidos WHERE cantidad <= 0 OR subtotal < 0;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
detalles_fuera_de_rango				
	0			

Evidencia de Etapa 3 – Consultas y reportes.

Referencia: (ver 05_consultas.sql)

Listado de los productos más vendidos por categoría.

```
6 • SELECT p.id as producto_id, p.nombre as producto, SUM(dp.cantidad) as cantidad_vendida, c.nombre as categoria
7 FROM productos p
8 INNER JOIN categorias c ON p.categoria_id = c.id
9 INNER JOIN detalles_pedidos dp ON dp.producto_id = p.id
10 group by p.id, p.nombre, c.nombre
11 ORDER BY cantidad_vendida DESC;
```

	producto_id	producto	cantidad_vendida	categoria
▶	334	Producto Demo 334	65	Bebidas
	450	Producto Demo 450	62	Bebidas
	910	Producto Demo 910	62	Bebidas
	850	Producto Demo 850	60	Bebidas
	54	Producto Demo 54	59	Bebidas
	142	Producto Demo 142	59	Bebidas
	566	Producto Demo 566	59	Bebidas
	262	Producto Demo 262	58	Bebidas
	738	Producto Demo 738	58	Bebidas
	970	Producto Demo 970	58	Bebidas
	150	Producto Demo 150	57	Bebidas
	246	Producto Demo 246	57	Bebidas
	306	Producto Demo 306	57	Bebidas
	362	Producto Demo 362	57	Bebidas

Listar los pedidos por usuario con su detalle.

```
18 • SELECT u.nombre as usuario,
19 po.nombre as nombre_producto,
20 p.estado as estado_pedido,
21 p.forma_pago as forma_pago,
22 dp.cantidad as cantidad,
23 dp.precio_unitario as precio
24 FROM pedidos p
25 inner JOIN usuarios u ON p.usuario_id = u.id
26 inner JOIN detalles_pedidos dp ON dp.pedido_id = p.id
27 inner JOIN productos po on dp.producto_id = po.id
28 order by u.nombre;
```

	usuario	nombre_producto	estado_pedido	forma_pago	cantidad	precio
▶	Ana Gomez	Producto Demo 16	PENDIENTE	TARJETA	2	20.07
	Ana Gomez	Producto Demo 17	PENDIENTE	TARJETA	4	17.84
	Ana Gomez	Producto Demo 18	PENDIENTE	TARJETA	4	2.32
	Ana Gomez	Producto Demo 37	CANCELADO	EFFECTIVO	2	19.26
	Ana Gomez	Producto Demo 38	CANCELADO	EFFECTIVO	2	5.46
	Ana Gomez	Producto Demo 58	PENDIENTE	EFFECTIVO	4	24.75
	Ana Gomez	Producto Demo 79	TERMINADO	TRANSFERENCIA	2	48.98
	Ana Gomez	Producto Demo 80	TERMINADO	TRANSFERENCIA	5	18.77
	Ana Gomez	Producto Demo 81	TERMINADO	TRANSFERENCIA	1	4.83
	Ana Gomez	Producto Demo 82	TERMINADO	TRANSFERENCIA	5	11.81
	Carlos Admin	Producto Demo 23	PENDIENTE	TARJETA	4	49.69
	Carlos Admin	Producto Demo 24	PENDIENTE	TARJETA	2	48.41
	Carlos Admin	Producto Demo 25	PENDIENTE	TARJETA	3	22.59

Listar los clientes con más frecuencia y alta facturación.

```

35 • SELECT u.id,
36       u.nombre as Nombre,
37       SUM(p.total) as total_facturado
38   from usuarios u
39  join pedidos p on u.id = p.usuario_id
40 where p.estado in ('CONFIRMADO', 'TERMINADO')
41 group by u.id, u.nombre
42 having total_facturado > 1000
43 order by total_facturado desc;
44

```

id	Nombre	total_facturado
182	Usuario Demo 179	1239.96
769	Usuario Demo 766	1182.54
801	Usuario Demo 798	1167.94
937	Usuario Demo 934	1166.04
804	Usuario Demo 801	1122.77
425	Usuario Demo 422	1120.11
890	Usuario Demo 887	1070.69
629	Usuario Demo 626	1051.02
407	Usuario Demo 404	1034.10
961	Usuario Demo 958	1025.64
257	Usuario Demo 254	1015.14
535	Usuario Demo 532	1008.54
351	Usuario Demo 348	1002.54

result 3 x

Listado del producto que tiene el stock por debajo del promedio.

```

48 • select id, nombre, stock
49   from productos
50  where stock < (select AVG(stock) from productos)
51  order by stock asc;
52

```

id	nombre	stock
109	Producto Demo 109	0
220	Producto Demo 220	0
940	Producto Demo 940	0
71	Producto Demo 71	1
869	Producto Demo 869	1
351	Producto Demo 351	2
446	Producto Demo 446	2
463	Producto Demo 463	2
645	Producto Demo 645	2
729	Producto Demo 729	2
129	Producto Demo 129	3
183	Producto Demo 183	3
979	Producto Demo 979	3
9	Producto Demo 9	4
60	Producto Demo 60	4
369	Producto Demo 369	4

productos 4 x

Medición comparativa con/sin índice.

Cuadro comparativo de tiempo de ejecución con y sin índice.

Consulta / Escenario	SIN ÍNDICE (Segundos)				CON ÍNDICE (Segundos)			
	Corrida 1	Corrida 2	Corrida 3	MEDIANA	Corrida 1	Corrida 2	Corrida 3	MEDIANA
LISTADO DE USUARIOS CONDICION POR EL ESTADO	0,0016 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg
CONSULTA SOBRE LA VISTA, COMO CONDICION EL EMAIL	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg
LISTADO DE PRODUCTOS, COMO CONDICION STOCK	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg	0,0000 seg

La implementación de índices en la base de datos, demuestra ser una estrategia para optimizar las consultas, logrando reducir el tiempo de ejecución. Aunque cuando las tablas tienen pocos registros no se ve esta diferencia en el tiempo.

Referencia: (ver 04_indices.sql y 05_explain.sql)

Primer Consulta.

- Sin INDICE (1°, 2° y 3° Corrida)

The screenshot shows a SQL query in an IDE. The query is:


```

    5 * EXPLAIN ANALYZE
    6 SELECT u.nombre, SUM(p.total)
    7 FROM usuarios u
    8 JOIN pedidos p ON u.id = p.usuario_id
    9 WHERE p.estado = 'TERMINADO'
    10 GROUP BY u.nombre;
    
```

 The 'Output' pane shows the execution plan:


```

    # Time Action Message Duration / Fetch
    1 15:15:29 use food_store 0 row(s) affected 0.000 sec
    2 15:15:36 EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p ON u.id = p.usuario_id WHERE p.estado = 'TERMINADO' GROUP BY u.nombre; 1 row(s) returned 0.016 sec / 0.000 sec
    3 15:18:22 EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p ON u.id = p.usuario_id WHERE p.estado = 'TERMINADO' GROUP BY u.nombre; 1 row(s) returned 0.000 sec / 0.000 sec
    4 15:20:01 EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p ON u.id = p.usuario_id WHERE p.estado = 'TERMINADO' GROUP BY u.nombre; 1 row(s) returned 0.000 sec / 0.000 sec
    
```

- Con índice. (1°, 2° y 3° Corrida)

The screenshot shows the same SQL query as above, but with the addition of a forced index:


```

    14 * EXPLAIN ANALYZE
    15 SELECT u.nombre, SUM(p.total)
    16 FROM usuarios u
    17 JOIN pedidos p FORCE INDEX (idx_pedidos_estado) ON u.id = p.usuario_id
    18 WHERE p.estado = 'TERMINADO'
    19 GROUP BY u.nombre;
    
```

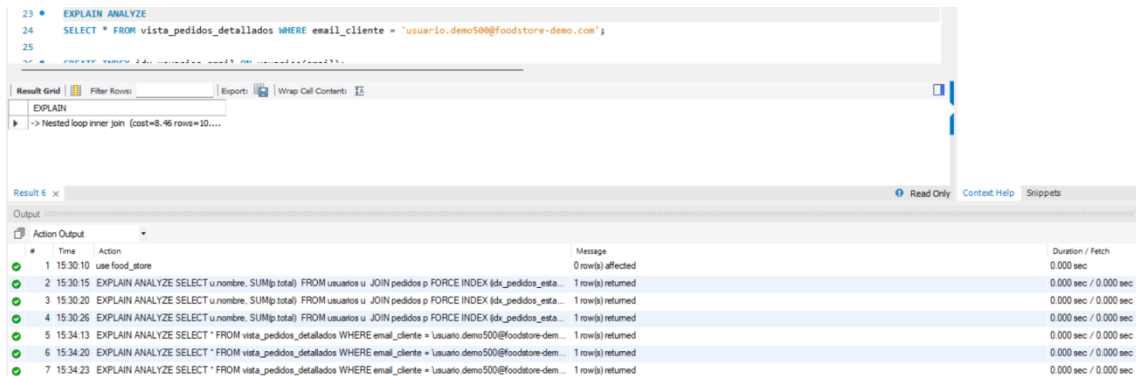
 The 'Output' pane shows the execution plan:


```

    # Time Action Message Duration / Fetch
    1 15:30:10 use food_store 0 row(s) affected 0.000 sec
    2 15:30:15 EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_estado) ON u.id = p.usuario_id WHERE p.estado = 'TERMINADO' GROUP BY u.nombre; 1 row(s) returned 0.000 sec / 0.000 sec
    3 15:30:20 EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_estado) ON u.id = p.usuario_id WHERE p.estado = 'TERMINADO' GROUP BY u.nombre; 1 row(s) returned 0.000 sec / 0.000 sec
    4 15:30:26 EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_estado) ON u.id = p.usuario_id WHERE p.estado = 'TERMINADO' GROUP BY u.nombre; 1 row(s) returned 0.000 sec / 0.000 sec
    
```

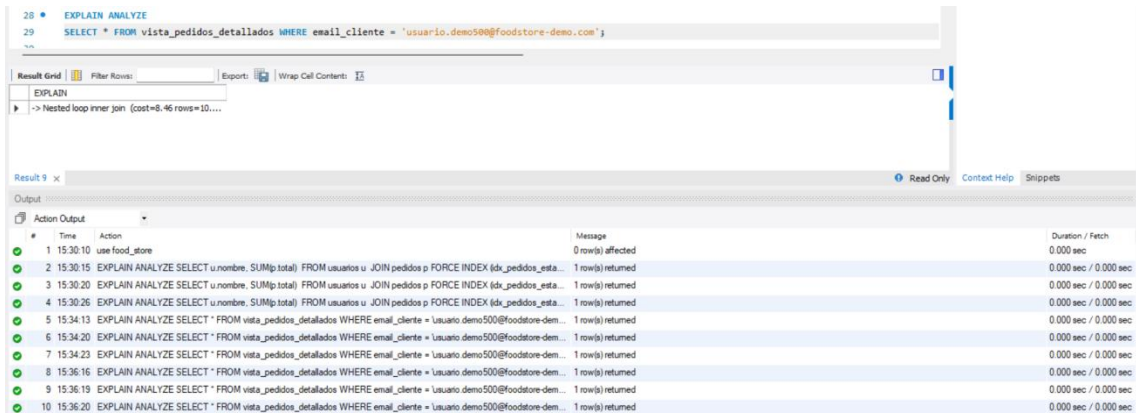
Segunda Consulta.

- Sin índice.
(1°, 2° y 3° Corrida)



#	Time	Action	Message	Duration / Fetch
1	15:30:10	use food_store	0 row(s) affected	0.000 sec
2	15:30:15	EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_esta...	1 row(s) returned	0.000 sec / 0.000 sec
3	15:30:20	EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_esta...	1 row(s) returned	0.000 sec / 0.000 sec
4	15:30:26	EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_esta...	1 row(s) returned	0.000 sec / 0.000 sec
5	15:34:13	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec
6	15:34:20	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec
7	15:34:23	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec

- Con índice
(1°, 2° y 3° Corrida)



#	Time	Action	Message	Duration / Fetch
1	15:30:10	use food_store	0 row(s) affected	0.000 sec
2	15:30:15	EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_esta...	1 row(s) returned	0.000 sec / 0.000 sec
3	15:30:20	EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_esta...	1 row(s) returned	0.000 sec / 0.000 sec
4	15:30:26	EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_esta...	1 row(s) returned	0.000 sec / 0.000 sec
5	15:34:13	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec
6	15:34:20	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec
7	15:34:23	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec
8	15:36:16	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec
9	15:36:19	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec
10	15:36:20	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario.demo500@foodstore-dem...	1 row(s) returned	0.000 sec / 0.000 sec

Tercer Consulta.

- Sin índice.
(1°, 2° y 3° Corrida)

33 • EXPLAIN ANALYZE
34 SELECT * FROM productos WHERE stock < 10;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

EXPLAIN
-> Index range scan on productos using idx_pr...

Result 12 x | Read Only | Context Help | Snippets

Output

#	Time	Action	Message	Duration / Fetch
4	15:30:26	EXPLAIN ANALYZE SELECT u.nombre, SUM(p.total) FROM usuarios u JOIN pedidos p FORCE INDEX (idx_pedidos_est...	1 row(s) returned	0.000 sec / 0.000 sec
5	15:34:13	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
6	15:34:20	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
7	15:34:23	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
8	15:36:16	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
9	15:36:19	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
10	15:36:20	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
11	15:38:16	EXPLAIN ANALYZE SELECT * FROM productos WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec
12	15:38:20	EXPLAIN ANALYZE SELECT * FROM productos WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec
13	15:38:21	EXPLAIN ANALYZE SELECT * FROM productos WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec

- Con índice.
(1°, 2° y 3° Corrida)

38 • EXPLAIN ANALYZE
39 SELECT * FROM productos FORCE INDEX (idx_productos_stock) WHERE stock < 10;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

EXPLAIN
-> Index range scan on productos using idx_pr...

Result 15 x | Read Only | Context Help | Snippets

Output

#	Time	Action	Message	Duration / Fetch
7	15:34:23	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
8	15:36:16	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
9	15:36:19	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
10	15:36:20	EXPLAIN ANALYZE SELECT * FROM vista_pedidos_detalle WHERE email_cliente = 'usuario demo500@foodstore-de...	1 row(s) returned	0.000 sec / 0.000 sec
11	15:38:16	EXPLAIN ANALYZE SELECT * FROM productos WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec
12	15:38:20	EXPLAIN ANALYZE SELECT * FROM productos WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec
13	15:38:21	EXPLAIN ANALYZE SELECT * FROM productos WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec
14	15:39:10	EXPLAIN ANALYZE SELECT * FROM productos FORCE INDEX (idx_productos_stock) WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec
15	15:39:11	EXPLAIN ANALYZE SELECT * FROM productos FORCE INDEX (idx_productos_stock) WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec
16	15:39:13	EXPLAIN ANALYZE SELECT * FROM productos FORCE INDEX (idx_productos_stock) WHERE stock < 10	1 row(s) returned	0.000 sec / 0.000 sec

Evidencia de Etapa 4 – Seguridad e integridad.

Referencia: (ver 06_vistas.sql y 07_seguridad.sql)

Verificación de permisos otorgados.

55 • SHOW GRANTS FOR 'operador_foodstore'@'localhost';
56

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Grants for operador_foodstore@localhost

▶	GRANT USAGE ON *.* TO 'operador_foodstore'@'localhost'
	GRANT SELECT ON 'food_store', 'vw_productos_disponibles' TO 'operador_foodstore'@'localhost'
	GRANT SELECT ON 'food_store', 'vw_usuarios_publico' TO 'operador_foodstore'@'localhost'

Prueba de Integridad: Violación de Clave primaria, violación de clave foránea y violación de restricción CHECK.

Con prueba anti-inyección documentada.

```
86 -- [CAPTURA A] PRUEBA DE INTEGRIDAD: Violación de Clave Primaria (PK Duplicada)
87 -- (Debe fallar con Error 1062: Duplicate entry '1' for key 'PRIMARY')
88 • INSERT INTO categorias (id, nombre, eliminado) VALUES (1, 'Bebidas Nuevas', 0);
89 -- [CAPTURA B] PRUEBA DE INTEGRIDAD: Violación de Clave Foránea (FK Inexistente)
90 -- (Debe fallar con Error 1452: Cannot add or update a child row: a foreign key constraint fails)
91 • INSERT INTO productos (nombre, precio, stock, categoria_id, eliminado) VALUES ('Producto Fantasma', 500.00, 10, 999, 0);
92 -- [CAPTURA C] PRUEBA DE INTEGRIDAD: Violación de Restricción CHECK (Precio Negativo)
93 -- (Debe fallar con Error 3819: Check constraint 'chk_producto_precio' is violated)
94 • INSERT INTO productos (nombre, precio, stock, categoria_id, eliminado) VALUES ('Producto Gratis', -100.00, 5, 1, 0);
95 -- [CAPTURA D] PRUEBA ANTI-INYECCIÓN DOCUMENTADA
96 -- (Esta da tilde VERDE y devuelve una grilla vacía de forma segura)
97 • CALL sp_buscar_producto_seguro('' OR ''1'='1');
```

#	Time	Action	Message
28	22:07:26	INSERT INTO categorias (id, nombre, eliminado) VALUES (1, 'Bebidas Nuevas', 0)	Error Code: 1062. Duplicate entry '1' for key 'categorias PRIMARY'
29	22:07:33	INSERT INTO productos (nombre, precio, stock, categoria_id, eliminado) VALUES ('Producto Fantasma', 500.00, 10, 999, 0)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('Food_store', 'pr...
30	22:07:39	INSERT INTO productos (nombre, precio, stock, categoria_id, eliminado) VALUES ('Producto Gratis', -100.00, 5, 1, 0)	Error Code: 3819. Check constraint 'chk_producto_precio' is violated.
31	22:07:44	CALL sp_buscar_producto_seguro('' OR ''1'='1')	0 row(s) returned

Validacion de mitigación de Inyeccion SQL.

Se sometió al procedimiento seguro a una rutina de ataque por fuerza bruta lógica inyectando la secuencia clásica de evasión de parámetros en texto plano.

```
01 • INSERT INTO categorias (id, nombre, eliminado) VALUES (1, 'Bebidas Nuevas', 0);
02
03
04 -- [CAPTURA B] PRUEBA DE INTEGRIDAD: Violación de Clave Foránea (fk Inexistente)
05 -- (Debe fallar con Error 1452: Cannot add or update a child row: a foreign key constraint fails)
06 • INSERT INTO productos (nombre, precio, stock, categoria_id, eliminado) VALUES ('Producto Fantasma', 500.00, 10, 999, 0);
07
08
09 -- [CAPTURA C] PRUEBA DE INTEGRIDAD: Violación de Restricción CHECK (Precio Negativo)
10 -- (Debe fallar con Error 3819: Check constraint 'chk_producto_precio' is violated)
11 • INSERT INTO productos (nombre, precio, stock, categoria_id, eliminado) VALUES ('Producto Gratis', -100.00, 5, 1, 0);
12
13
14 -- [CAPTURA D] PRUEBA ANTI-INYECCIÓN DOCUMENTADA
15 -- (Esta da tilde VERDE y devuelve una grilla vacía de forma segura)
16 • CALL sp_buscar_producto_seguro('' OR ''1'='1');
```

#	Time	Action	Response	Duration / Fetch Time
1	11:12:17	INSERT INTO categorias (id, nombre, eliminado) VALUES (1, 'Bebidas Nuevas', 0)	Error Code: 1062. Duplicate entry '1' for key 'categorias...	0.00036 sec
2	11:12:21	INSERT INTO productos (nombre, precio, stock, categoria_id, eliminado) VALUES ('Producto Fantasma', 500.00, 10, 999, 0)	Error Code: 1452. Cannot add or update a child row: a...	0.00018 sec
3	11:13:50	INSERT INTO productos (nombre, precio, stock, categoria_id, eliminado) VALUES ('Producto Gratis', -100.00, 5, 1, 0)	Error Code: 3819. Check constraint 'chk_producto_pri...	0.00029 sec
4	11:14:16	CALL sp_buscar_producto_seguro('' OR ''1'='1')	0 row(s) returned	0.00020 sec / 0.00001...

Evidencia de Etapa 5 – Concurrencia y transacciones.

Referencia: (ver 08_transacciones.sql y 09_concurrencia_guiada.sql)

Pedido pendiente con stock suficiente.

```
145 • SELECT * FROM log_transacciones WHERE pedido_id = @pedido_test_ok ORDER BY id DESC;
146 • SELECT id, estado FROM pedidos WHERE id = @pedido_test_ok;
147
148 -- =====
```

	id	pedido_id	intento	resultado	detalle	creado_en
▶	1	2	1	OK	Pedido confirmado correctamente	2026-06-22 22:14:45
*		NULL	NULL	NULL	NULL	NULL

ERROR 1644 (45000) Stock insuficiente para confirmar el pedido

```
168 • CALL sp_confirmar_pedido(@pedido_test_fail);
```

#	Time	Action	Message
45	22:17:00	UPDATE productos SET stock = 0 WHERE id = @producto_fail	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
46	22:17:04	CALL sp_confirmar_pedido(@pedido_test_fail)	Error Code: 1644, Stock insuficiente para confirmar el pedido

Los movimientos asociados al pedido que falló, ordenados desde el más reciente al más antiguo.

```
170 • SELECT * FROM log_transacciones WHERE pedido_id = @pedido_test_fail ORDER BY id DESC;
```

	id	pedido_id	intento	resultado	detalle	creado_en
▶	2	3	1	STOCK_INSUFICIENTE	Producto 23 no tiene stock suficiente para la ca...	2026-06-22 22:17:04
*		NULL	NULL	NULL	NULL	NULL

Estado actual del pedido.

```
171 • SELECT id, estado FROM pedidos WHERE id = @pedido_test_fail;
172
```

	id	estado
▶	3	PENDIENTE
*		NULL

Concurrencia guiada.

PASO 0 (cualquier sesión)

```
23 • SELECT id, nombre, stock
24 FROM productos
25 WHERE stock > 50
26 ORDER BY id
27 LIMIT 4;
```

Result Grid			
Filter Rows:			
Edit:			
#	id	nombre	stock
1	1	Producto Demo 1	193
2	2	Producto Demo 2	91
3	3	Producto Demo 3	189
4	4	Producto Demo 4	182
*	NULL	NULL	NULL

PARTE 1 — Simulación de un deadlock real con dos sesiones

SQL File 5*			
SQL File 3*			
SQL File 4*			
SQL File 5*			
Limit to 1000 rows			
1	COMMIT;		
2			
3 •	SELECT id, stock FROM productos WHERE id IN (1,2);		

Result Grid		
Filter Rows:		
Edit:		
#	id	stock
1	1	185
2	2	87
*	NULL	NULL

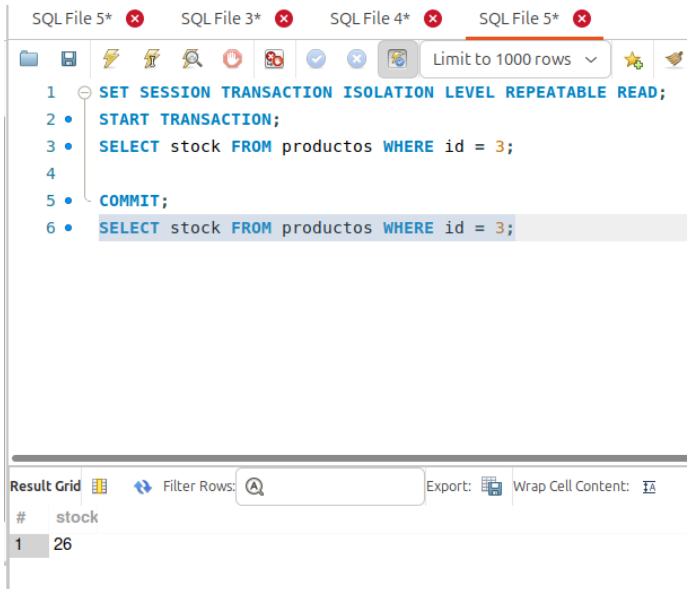
PARTE 2 — Comparación de niveles de aislamiento

Ejecutando la Sesión A en su primera lectura:

SQL File 5*			
SQL File 3*			
SQL File 4*			
SQL File 5*			
Limit to 1000 rows			
1	SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;		
2 •	START TRANSACTION;		
3 •	SELECT stock FROM productos WHERE id = 3;		

Result Grid	
Filter Rows:	
Export:	
Wrap Cell Content:	
#	stock
1	76

Se ejecuta el Paso B1 (en la otra sesión) y se vuelve a leer en la Sesión A (Paso A2).

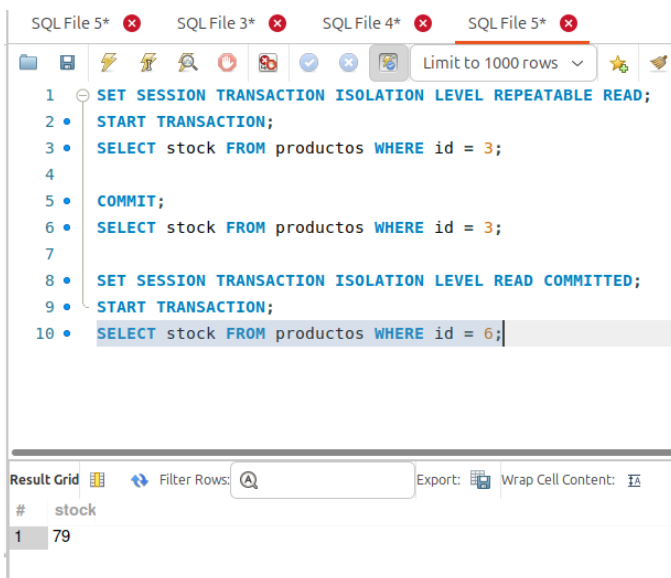


The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are four tabs labeled 'SQL File 5*', 'SQL File 3*', 'SQL File 4*', and 'SQL File 5*'. The active window displays the following SQL code:

```
1 SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
2 START TRANSACTION;  
3 SELECT stock FROM productos WHERE id = 3;  
4  
5 COMMIT;  
6 SELECT stock FROM productos WHERE id = 3;
```

Below the code editor, the 'Result Grid' is visible. It shows a single column header 'stock' and one row with the value '26'.

Cierra la transacción y vuelve a leer



The screenshot shows the same SQL Server Enterprise Manager interface. The active window displays the following SQL code:

```
1 SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
2 START TRANSACTION;  
3 SELECT stock FROM productos WHERE id = 3;  
4  
5 COMMIT;  
6 SELECT stock FROM productos WHERE id = 3;  
7  
8 SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;  
9 START TRANSACTION;  
10 SELECT stock FROM productos WHERE id = 6;
```

Below the code editor, the 'Result Grid' is visible. It shows a single column header 'stock' and one row with the value '79'.

READ COMMITTED

SQL File 5* SQL File 3* SQL File 4* SQL File 5*

Limit to 1000 rows

```
1 SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
2 START TRANSACTION;
3 SELECT stock FROM productos WHERE id = 3;
4
5 COMMIT;
6 SELECT stock FROM productos WHERE id = 3;
7
8 SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
9 START TRANSACTION;
10 SELECT stock FROM productos WHERE id = 6;
11
```

Result Grid

Filter Rows: Export: Wrap Cell Content:

#	stock
1	29

Evidencia del uso de IA como tutoría.

Incorporamos asistentes de IA generativa como un recurso de apoyo y tutoría a lo largo de las cinco etapas de este proyecto. El objetivo primordial de esta integración fue potenciar el pensamiento crítico mediante la ingeniería de *prompts*.

En lugar de delegar la resolución del problema, el equipo planteó desafíos lógicos a la IA para recibir guías algorítmicas paso a paso, corregir razonamientos erróneos de normalización y explorar optimizaciones en la manipulación de datos masivos.

Anexo IA - Etapa 1: Modelado y Constraints.

Interacción 1: Estructuración del Script Idempotente

Prompt del Estudiante:

"Hola. Estoy diseñando la base de datos para el Trabajo Final Integrador de un sistema de gestión de pedidos llamado 'Food Store'. Necesito armar el script inicial '01_esquema.sql' en MySQL 8. Las entidades son Categoría, Producto, Usuario, Pedido y DetallePedido. El script tiene que ser idempotente, usar buenas prácticas, tener claves primarias, foráneas con ON DELETE RESTRICT, unicidad en mails/nombres y validaciones CHECK para precios y stocks que no sean negativos. ¿Cómo estructuramos el código DDL de forma profesional?"

Respuesta de la IA:

Para lograr un diseño robusto e idempotente en MySQL 8+, debemos estructurar el script eliminando las tablas en el orden inverso a sus dependencias jerárquicas (para evitar fallas de Clave Foránea) y luego creándolas respetando el orden relacional. Aquí tienes la propuesta técnica para el esquema:

```
CREATE DATABASE IF NOT EXISTS food_store;  
USE food_store;
```

```
DROP TABLE IF EXISTS detalle_pedido;  
DROP TABLE IF EXISTS pedido;  
DROP TABLE IF EXISTS producto;  
DROP TABLE IF EXISTS usuario;  
DROP TABLE IF EXISTS categoria;
```

```
-- Tabla Categoría  
CREATE TABLE categoria (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL UNIQUE,
```

```

    descripcion TEXT,
    eliminado BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla Producto
CREATE TABLE producto (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    categoria_id BIGINT NOT NULL,
    nombre VARCHAR(150) NOT NULL UNIQUE,
    precio DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL,
    disponible BOOLEAN NOT NULL DEFAULT TRUE,
    eliminado BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_producto_categoria FOREIGN KEY (categoria_id)
        REFERENCES categoria(id) ON DELETE RESTRICT,
    CONSTRAINT chk_producto_precio CHECK (precio >= 0),
    CONSTRAINT chk_producto_stock CHECK (stock >= 0)
);

```

Interacción 2: Estrategia de Borrado Lógico (Soft Delete) e Integridad Referencial

Prompt del Estudiante:

"En la materia de Programación nos exigen que usemos Soft Delete (baja lógica con un atributo 'eliminado'), por lo que nunca borramos físicamente de las colecciones. ¿Cómo se traduce esto correctamente a la base de datos y por qué elegimos ON DELETE RESTRICT en las claves foráneas?"

Respuesta de la IA:

El uso de ON DELETE RESTRICT es la contraparte perfecta para una arquitectura orientada a Soft Delete. Al definir RESTRICT, el motor de la base de datos bloqueará activamente cualquier intento de ejecución de un comando DELETE FROM físico si la entidad padre está asociada a registros hijos (por ejemplo, intentar borrar una Categoría que tiene Productos cargados). En su lugar, el sistema se ve obligado a realizar un UPDATE tabla SET eliminado = true WHERE id = X;. Al agregar la columna eliminado BOOLEAN DEFAULT FALSE en todas las tablas primarias, mantienes el historial de transacciones intacto (esencial para auditorías de pedidos antiguos) sin romper las restricciones del modelo relacional.

Interacción 3: Diseño de Pruebas de Integridad de Datos (Casos de Éxito y Fallo)

Prompt del Estudiante:

"Para el entregable nos piden documentar en el PDF de BD-I un cuadro con 2 inserciones válidas y 2 inválidas detallando los mensajes de error del motor. ¿Me ayudás a plantear los casos de prueba para verificar que los CHECK y UNIQUE funcionen?"

Respuesta de la IA:

Para demostrar la robustez de tus restricciones en el informe, puedes documentar los siguientes escenarios de prueba DML dentro de tu archivo de catálogo:

-- [CASO VÁLIDO 1] Inserción correcta de Categoría

```
INSERT INTO categoria (nombre, descripcion) VALUES ('Hamburguesas', 'Variedad de medallones de carne y pollo');
```

-- [CASO INVÁLIDO 1] Violación de Restricción UNIQUE (Nombre Duplicado)

-- Error esperado: ERROR 1062 (23000): Duplicate entry 'Hamburguesas' for key 'categoria.nombre'

```
INSERT INTO categoria (nombre, descripcion) VALUES ('Hamburguesas', 'Otra descripción diferente');
```

-- [CASO INVÁLIDO 2] Violación de Restricción CHECK (Precio Negativo)

-- Error esperado: ERROR 3819 (HY000): Check constraint 'chk_producto_precio' is violated.

```
INSERT INTO producto (categoria_id, Product_name, precio, stock) VALUES (1, 'Doble con Queso', -450.00, 20);
```

Anexo IA — Etapa 2 (Datos masivos).

Prompt del Estudiante:

"se le pidió al asistente diseñar 03_carga_masiva.sql y 04_indices.sql para el sistema Food Store, cumpliendo con el requisito de ≥ 10.000 registros totales, FKs siempre válidas y al menos un detalle por pedido, reutilizando el esquema ya definido por el Integrante 1 en 01_esquema.sql."

Respuesta de la IA:

- ☐ Usar WITH RECURSIVE para generar series numéricas 1..N en lugar de tablas de números auxiliares persistentes, manteniendo el script autocontenido.
- ☐ Combinar esas series con ROW_NUMBER() OVER (ORDER BY id) sobre las tablas semilla (categorias, usuarios, productos) y distribuir las FKs con el operador módulo, evitando así cualquier id inventado.
- ☐ Generar pedidos y detalles_pedidos en dos pasos: primero los pedidos con total = 0, luego los detalles, y finalmente un UPDATE ... JOIN que recalcula el total real sumando los subtotales — así nunca queda un pedido con total inconsistente.

- Garantizar cardinalidad mínima (cada pedido con al menos 1 detalle) con la condición $c.k \leq (p.id \% 4) + 1$ en el cross join, en vez de un filtro aleatorio que podría dejar pedidos sin detalle.

Error detectado y corregido durante las pruebas: Al ejecutar el script contra una instancia real de MySQL 8.0 (contenedor Docker levantado para validar el trabajo), MySQL rechazó la sintaxis `WITH RECURSIVE ... INSERT INTO tabla SELECT ...` con el error 1064. La IA identificó que, a diferencia de otros motores, MySQL exige que la cláusula `WITH` se ubique **después** de `INSERT INTO tabla (columnas)` y **antes** del `SELECT`, no al inicio del statement. Se corrigieron las cuatro inserciones masivas con ese orden y el script pasó a ejecutarse sin errores.

Validación de resultados: se volvió a ejecutar el pipeline completo (`01_esquema.sql` → `02_catalogo.sql` → `03_carga_masiva.sql` → `04_indices.sql`) contra MySQL 8.0 real. Se confirmó: 16.003 registros totales, 0 FKs huérfanas, 0 pedidos sin detalle, 0 valores fuera de rango, y que `pedidos.total` coincide exactamente con la suma de los `subtotal` de cada pedido. También se verificó que `04_indices.sql` es idempotente (ejecutarlo dos veces no falla ni duplica índices), usando un procedimiento almacenado que consulta `information_schema.statistics` antes de cada `CREATE INDEX`, ya que MySQL no soporta `DROP INDEX IF EXISTS`.

Decisiones que se tomaron por criterio propio (no sugeridas por la IA).

- Los volúmenes concretos por tabla (1.000 usuarios, 1.000 productos, 4.000 pedidos, ~10.000 detalles) se eligieron para superar cómodamente el mínimo exigido sin generar un script lento de ejecutar en clase.
- Los índices elegidos en `04_indices.sql` se basaron en los joins y filtros que se anticipan para las consultas de Etapa 3 (FKs de `productos`, `pedidos` y `detalles_pedidos`, filtro por `estado` y por `created_at`).

Anexo IA — Etapa 3 (Consultas complejas).

Prompt del Estudiante:

“Teniendo una base de datos con de un sistema de gestión de pedidos teniendo como entidades Categoría, Producto, Usuario, Pedido y DetallePedido, en mi caso tuve que realizar consultas complejas, en mi primer consulta que hice relacione producto, categoría y detalle utilizando group by y ordenando la cantidad, el objetivo es listar los productos más vendidos según la categoría. ¿Como podría mejorar esta consulta?”

Respuesta de la IA:

🔗 **JOIN explícitos:** asegúrate de usar `INNER JOIN` o `LEFT JOIN` según corresponda, en lugar de depender de condiciones en el `WHERE`.

🔗 **SUM en lugar de COUNT:** si en `DetallePedido` tienes la columna `cantidad`, conviene sumar esa cantidad en vez de contar filas, porque un producto puede repetirse en varios pedidos.

🔗 **CTE o subconsulta:** usar una CTE (`WITH`) puede hacer más clara la lógica, especialmente si luego quieres filtrar por el top N de cada categoría.

🔗 **Funciones de ventana:** si quieres obtener el producto más vendido por categoría, puedes usar ROW_NUMBER() o RANK() en lugar de un GROUP BY con ORDER BY.

Prompt del Estudiante:

“Dada esta consulta diseñada como sería posible mejorarla”.

Respuesta de la IA:

🔗 **Usar SUM en lugar de COUNT:** ya lo aplicaste correctamente (SUM(dp.cantidad)), lo cual refleja mejor las ventas reales.

🔗 **Evitar SELECT con columnas no agrupadas:** tu GROUP BY está bien porque incluye todas las columnas seleccionadas que no son agregadas.

🔗 **Funciones de ventana:** si quieres obtener el producto más vendido por categoría, puedes usar ROW_NUMBER() o RANK() en lugar de solo ordenar globalmente.

🔗 **Filtrar por categoría:** si te interesa una categoría específica, agrega WHERE c.id = ... para reducir el costo de la consulta.

🔗 **CTE para claridad:** encapsular la agregación en una CTE hace más legible el código y facilita extenderlo.

Anexo IA - Etapa 4: Seguridad, Vistas e Inyección SQL.

Interacción 1: Sincronización Estructural y Errores de Nomenclatura DDL

Prompt del Estudiante:

"Hola. Estoy programando el script de seguridad 07_seguridad.sql para el proyecto Food Store, pero al compilar las vistas que ocultan el borrado lógico y las contraseñas, el motor me tira un Error 1146 diciendo que la tabla tpi.usuarios o las columnas no existen. Les paso el script base 01_esquema.sql para revisar la nomenclatura real. ¿Cómo adaptamos las vistas al estándar que ya tenemos aprobado?"

Respuesta de la IA:

El error radica en un desacople entre los nombres de prueba y el esquema DDL unificado. Analizando el archivo base 01_esquema.sql, se constata que:

1. La base de datos definitiva se denomina food_store y no tpi.
2. Las tablas principales se definieron en plural (usuarios, productos, categorias).
3. Las claves primarias se denominan estrictamente id y la relación foránea es categoria_id.

Redefiniendo las vistas bajo tus estándares exactos, la sintaxis segura e idempotente en MySQL 8+ se debe estructurar de la siguiente manera:

```
CREATE OR REPLACE VIEW food_store.vw_usuarios_publico AS
```

```
SELECT id, nombre, mail, rol, creado_at
```

```
FROM food_store.usuarios;
```

Interacción 2: Parametrización y Blindaje de Consultas Dinámicas (Inyección SQL)

Prompt del Estudiante:

"El profesor de la UTN nos pide incorporar un procedimiento almacenado de búsqueda que demuestre protección total contra inyección SQL. Si un atacante ingresa una cadena maliciosa como ' OR '1'='1, ¿cómo se asegura el motor de que se procese como texto y no como código ejecutable?"

Respuesta de la IA:

Para neutralizar un ataque de inyección SQL sin recurrir a sentencias preparadas dinámicas complejas (las cuales abren vulnerabilidades si se concatenan directo), aprovechamos la parametrización nativa de los Procedimientos Almacenados en MySQL.

Al definir un parámetro de entrada IN p_nombre_buscar VARCHAR(100), el SGBD precompila el plano de ejecución. El uso de la función CONCAT('%', p_nombre_buscar, '%') asegura que cualquier comilla simple o instrucción lógica inyectada sea tratada estrictamente como una cadena de caracteres literal, devolviendo un conjunto vacío de resultados seguros en lugar de comprometer las tablas.

Anexo IA — Etapa 5 (Concurrencia y Transacciones)

Prompt del Estudiante:

"se le pidió al asistente revisar primero el estado general del TFI de Bases de Datos I contra la consigna oficial (Trabajo_Final_Integrador_Bases_de_Datos1.pdf) y el plan de equipo (Plan_TPI_Integrado.md), para confirmar qué faltaba concretamente de la Etapa 5 (responsabilidad de Cesia): los archivos 08_transacciones.sql y 09_concurrencia_guiada.sql no existían todavía."

Respuesta de la IA:

Estrategia propuesta por la IA para 08_transacciones.sql:

- ☐ Elegir como operación crítica "confirmar un pedido" (descuento de stock por cada detalle + cambio de estado), porque es el caso de negocio real del dominio Food Store que más se beneficia de una transacción correcta — en lugar de un ejemplo de transacción genérico sin relación con el sistema.

- ❑ Usar un procedimiento almacenado con cursor sobre `detalles_pedidos`, bloqueando cada fila involucrada con `SELECT ... FOR UPDATE` (sobre el pedido y sobre cada producto) para serializar el acceso concurrente al stock.
- ❑ Declarar un manejador específico (`DECLARE EXIT HANDLER FOR SQLSTATE '40001'`) para el error de deadlock, en vez de un manejo de errores genérico, de modo que solo ese caso dispare el retry.
- ❑ Envolver la lógica en un `WHILE` con contador de intentos (máximo 3: el inicial más hasta 2 reintentos) y un `SLEEP()` creciente (0.3 s, 0.6 s) como backoff breve entre reintentos.
- ❑ Registrar cada resultado (`OK`, `DEADLOCK`, `STOCK_INSUFICIENTE`, `FALLO_DEFINITIVO`) en una tabla `log_transacciones` para poder auditarlo con un simple `SELECT`, en lugar de depender únicamente de los mensajes de error de la sesión.

Estrategia propuesta por la IA para `09_concurrencia_guiada.sql`:

- ❑ Escribir un guion paso a paso con marcadores explícitos de sesión (**SESIÓN A / SESIÓN B**) y de orden (A1, B1, A2, B2...), porque las variables de usuario (`@variable`) de MySQL no se comparten entre conexiones distintas: hace falta usar placeholders de id de producto que el alumno reemplaza a mano tras correr una consulta previa.
- ❑ Simular el deadlock con dos transacciones que toman locks sobre dos productos en orden cruzado (A bloquea el producto 1 y va por el 3; B bloquea el 3 y va por el 1), cerrando un ciclo de espera que InnoDB detecta y resuelve abortando una de las dos.
- ❑ Demostrar la diferencia entre `REPEATABLE READ` y `READ COMMITTED` con el ejemplo más simple posible: una sesión lee el mismo dato dos veces dentro de la misma transacción mientras la otra sesión lo modifica y confirma en el medio.

Validación real contra MySQL 8.0: para no entregar un script sin probar, se levantó un contenedor MySQL 8.0 descartable con Docker y se ejecutó la cadena completa (`01_esquema.sql` → `02_catalogo.sql` → `03_carga_masiva.sql` → `04_indices.sql` → `06_vistas.sql` → `08_transacciones.sql`) contra datos reales generados en la Etapa 2.

- ❑ **Camino feliz:** se confirmó un pedido `PENDIENTE` con stock suficiente; quedó en estado `CONFIRMADO` y se registró un log `OK`.
- ❑ **Camino de error:** se forzó el stock de un producto a 0 en otro pedido `PENDIENTE` distinto; el procedimiento hizo `ROLLBACK`, registró el log `STOCK_INSUFICIENTE` y el pedido quedó intacto en estado `PENDIENTE` — confirmando que no quedaron cambios parciales.

Validación del guion de dos sesiones: un único cliente `mysql` no permite abrir dos sesiones simultáneas a la vez, así que la IA armó un script Python (con `subprocess`) que abre dos conexiones concurrentes contra el mismo contenedor y envía los comandos de cada sesión en el orden indicado por el guion.

Problema detectado y corregido: al principio no se veía ninguna salida de las consultas, porque el cliente `mysql` usa buffering por bloques (no por línea) cuando su salida estándar no es una terminal interactiva, como pasa al correrlo dentro de un contenedor vía `docker exec`. Se resolvió ejecutando el cliente envuelto en `stdbuf -oL -eL` dentro del contenedor, lo que fuerza la salida línea por línea y permitió leer los resultados de cada sesión en tiempo real.

Resultado real obtenido (deadlock): se reprodujo el error `ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction` tal cual estaba previsto en el guion. En esta corrida la Sesión B fue la víctima elegida por InnoDB (no la A), confirmando que no es determinístico cuál de las dos sesiones se aborta.

Resultado real obtenido (aislamiento): con `REPEATABLE READ`, la segunda lectura de la Sesión A dentro de la misma transacción devolvió el mismo valor que la primera (136) aunque la Sesión B ya había confirmado un cambio a 86. Con `READ COMMITTED`, la segunda lectura sí reflejó de inmediato el cambio de la Sesión B (de 58 a 8). Ambos resultados coinciden exactamente con el comportamiento documentado en `Etapas_Descripcion_Conceptual.md`.

Decisiones que se tomaron por criterio propio (no sugeridas por la IA)

- ☐ El número máximo de reintentos (2, además del intento inicial) y los tiempos de backoff (0.3 s y 0.6 s) se fijaron en base al mínimo pedido por la consigna ("hasta 2 reintentos con backoff breve"), sin necesidad de ajustarlos con la IA.
- ☐ Los cuatro productos usados como casos de prueba (uno por cada bloque del guion) se eligieron a mano filtrando por stock alto, para que ninguna demostración fallara por quedarse sin stock antes de llegar al paso que importa observar.
- ☐ La decisión de no automatizar por completo el guion de dos sesiones, sino dejarlo como instrucciones para ejecutar a mano en dos pestañas de MySQL Workbench (con el script Python solo como verificación interna antes de entregar), porque la consigna pide explícitamente una simulación con dos sesiones reales.

Bibliografía.

[1] Cátedra UTN TUPaD. Material teórico de Base de Datos I –Disponible en: Campus virtual UTN (Aula virtual de la materia)

[2] Cátedra UTN TUPaD. Consigna del Trabajo Práctico Integrador – Base de Datos I (2026). Disponible en: Documento PDF provisto por la cátedra

[3] Google DeepMind. Gemini (Versión de lenguaje extendido). [Software de Inteligencia Artificial]. Disponible en: <https://gemini.google.com/>. Utilizado como tutor pedagógico y asistente de optimización de consultas SQL (2026).

[4] Anthropic. Claude (Modelo Claude 3.5 Sonnet / Claude Code). [Software de Inteligencia Artificial y CLI de asistencia]. Disponible en: <https://www.anthropic.com/claude>. Utilizado como herramienta de asistencia técnica, depuración de scripts de concurrencia y validación de planes de ejecución (2026).

Link de video.

A continuación se lista el enlace requeridos por la consigna. Cuentan con permisos públicos de visualización.

Recurso	URL
Video demostración (10–15 min)	https://drive.google.com/file/d/1n4R8U-EYZghgbk8jc6Zf6mpPgP0-asrq/view?usp=sharing